

Combining Instance Filtering and Attribute Weighting in Decision Tree and Naïve Bayes Hybrids

Ikramul Hasan Moral

Department of Computer Science and Engineering
United International University
Dhaka, Bangladesh
imoral223489@bscse.uiu.ac.bd

Dewan Md. Farid

Professor, Department of Computer Science and Engineering
United International University
Dhaka, Bangladesh
dewanfarid@cse.uiu.ac.bd

Abstract—Naïve Bayes (NB) can filter training instances before a decision tree is fitted, while a tree can select and weight attributes for NB. We combine these steps as Algorithm 3 and test both orders, with either NB or C4.5 as the final classifier. The evaluation uses ten benchmark datasets and ten repetitions of stratified 10-fold cross-validation. Every cleaning step is refitted within the training fold, and every held-out row is scored. Mean accuracy is 81.21% and 80.89% for the two combined tree methods, compared with 86.33% for uncleaned C4.5. The combined NB methods reach 75.63% and 76.35%, compared with 75.93% for plain NB and 77.35% for attribute weighting alone. Neither order shows a clear accuracy advantage in paired dataset-level tests. Fitting the instance filter before cross-validation raises its apparent accuracy by 13.50 percentage points, but also removes cases from the evaluation sample. Class-specific removal records show that the filter can discard a much larger share of one class. On these datasets, combining the steps does not establish an improvement over the base classifiers; the protocol and the discarded cases are central to interpreting the scores.

Index Terms—Naïve Bayes, decision tree, instance filtering, attribute weighting, hybrid classifier, cross-validation

I. INTRODUCTION

Farid et al. [1] introduced two hybrids that connect Naïve Bayes (NB) and decision trees. Algorithm 1 trains NB on the training set, removes the instances it misclassifies, and grows a tree on the remaining rows. Algorithm 2 grows a tree first, keeps the attributes it tests, and gives greater weight to attributes used nearer the root when fitting NB. Each method lets one classifier prepare data for the other.

Applying both steps is a natural extension, but their combination changes what each learner sees. If instances are removed first, the tree selects attributes from a smaller sample. If attributes are selected first, a weighted NB judge decides which instances remain. We call this combined procedure Algorithm 3 and examine both orders. A final tree and a final NB are evaluated separately so that a change of classifier does not stand in for evidence that cleaning helped.

The evaluation procedure also changes the meaning of a score. A filter fitted before cross-validation can inspect labels from future test folds and remove those cases before they are

evaluated. Fitting the filter inside each training fold leaves all held-out rows available for scoring. The first procedure therefore changes both the information used for cleaning and the population on which accuracy is measured.

We compare Algorithm 3 with the individual steps and uncleaned baselines under a shared, fold-specific protocol. The analysis covers accuracy, macro-F1, and class-specific deletion rates. It also contrasts cleaning inside the training fold with cleaning before the split. The combined methods do not show a clear advantage on this benchmark. Our contribution is the controlled comparison and its explanation through protocol effects and removal records.

II. RELATED WORK

Tree-based weighting for NB has an established history. Hall [2] used decision trees to derive attribute weights, addressing NB’s conditional-independence assumption. Farid et al. [1] proposed the two specific steps studied here and evaluated them separately. Algorithm 3 composes those steps while retaining their underlying tree learner and NB likelihood model.

Instance filtering requires a separate caution. Brodley and Friedl [3] studied the identification of mislabeled training data using several classifiers. A single NB judge’s disagreement with a supplied label cannot establish that the label is wrong. In this study, “filtered” means rejected by that judge, without a claim that the removed rows contain verified label errors.

Cawley and Talbot [4] showed how evaluation practices can introduce selection bias large enough to affect comparisons between learning algorithms. We examine the placement of supervised cleaning for this particular hybrid family. Dataset-level paired comparisons follow the framework discussed by Demšar [5]; repeated folds serve to estimate each dataset’s score rather than provide independent observations for the significance tests.

III. ALGORITHMS

A. Algorithm 1: instance filtering

Given training data D , the NB judge is fitted on D and predicts the labels of those same rows. A row remains only when the prediction agrees with its supplied label. Algorithm 1 then fits C4.5 on the retained rows. The reference arm ending in NB uses the same filter but refits NB as its final classifier. If deletion would leave fewer than two classes, our implementation retains the original training rows.

This is a training-set disagreement rule. Its predictions are not out-of-fold estimates of label quality. A judge can reject a correctly labeled case because the class distributions overlap or because its own model assumptions are unsuitable.

B. Algorithm 2: attribute selection and weighting

A C4.5 tree is fitted on the training data. Let d_j be the smallest depth at which it tests attribute j , with the root at depth one. The weight is

$$w_j = \begin{cases} 1/\sqrt{d_j}, & j \text{ is tested,} \\ 0, & \text{otherwise.} \end{cases} \quad (1)$$

Untested attributes are dropped. The weighted NB class score is

$$s_c(x) = \log P(c) + \sum_{j \in S} w_j \log P(x_j | c), \quad (2)$$

where S is the retained attribute set. NB predicts the class with the largest score. The reference arm ending in a tree uses the selected columns, but its final C4.5 does not use NB's attribute weights.

C. Algorithm 3: the combined hybrid

The instances-first path, denoted NA, runs Algorithm 1's plain NB filter and then derives the selected attributes and weights from a tree on the retained rows. The attributes-first path, AN, derives the attributes and weights from the original training fold, then uses weighted NB to filter its rows. In AN, the same weights reach both the judge and the final NB; they are not re-estimated after deletion. Each path ends in either weighted NB or C4.5.

Algorithm 3 uses the filtering safeguard stated above. If a selection tree tests no attribute, the combined path keeps all current columns with unit weights. No empty-selection event occurs in the stored runs. At prediction time, the test data keep the selected columns S and all their rows. Every held-out case must still be classified.

IV. EXPERIMENTAL METHOD

A. Datasets and implementation

We use the ten dataset names from the 2014 study. Table I reports the sizes of our available copies. Image segmentation combines the two available UCI files, and NSL-KDD uses its 20% training file [6]. The available Glass and NSL-KDD files contain one fewer class than the counts listed in the original study. These choices allow a comparison within a common

Algorithm 3 Combined cleaning and classification

Require: Training data D , order $o \in \{\text{NA}, \text{AN}\}$, final learner

```

 $L \in \{\text{NB}, \text{DT}\}$ 
1: if  $o = \text{NA}$  then
2:    $D \leftarrow \text{FILTER}(D, \text{plain NB})$ 
3:    $(S, W) \leftarrow \text{SELECT}(D, \text{C4.5})$ 
4: else
5:    $(S, W) \leftarrow \text{SELECT}(D, \text{C4.5})$ 
6:    $D \leftarrow D[:, S]$ 
7:    $D \leftarrow \text{FILTER}(D, \text{NB with } W)$ 
8: end if
9: if  $o = \text{NA}$  then
10:   $D \leftarrow D[:, S]$ 
11: end if
12: Fit  $M$  on  $D$  using  $L$  (use  $W$  if  $L = \text{NB}$ )
13: return  $M, S$ 

```

TABLE I
DATASET SIZES IN THE FILES USED FOR EVALUATION

Dataset	Rows	Attributes	Classes
Breast cancer	286	9	2
Contact lenses	24	4	3
Diabetes	768	8	2
Glass	214	9	6
Iris	150	4	3
Soybean	683	35	19
Vote	435	16	2
Image segmentation	2310	19	7
Tic-tac-toe	958	9	2
NSL-KDD	25192	41	22

benchmark family, but do not establish an exact reproduction of the original data split.

C4.5 is Weka J48 version 3.8.6 with pruning enabled and defaults `-C 0.25 -M 2` [7]. Our NB implementation uses add-one counts for nominal attributes and Gaussian likelihoods for numeric attributes. Nominal attributes remain single columns. A tree therefore selects an original attribute rather than individual one-hot indicators. Category vocabularies are taken from the available dataset files and held fixed across folds. Missing nominal values are represented by a separate category. Counts, numeric distributions, and class priors are estimated from training rows.

B. Protocols and comparison arms

Protocol B is the main evaluation. We run stratified 10-fold cross-validation with seeds 0–9, giving 100 scored folds per dataset and method. All methods share each split. The judge, selection tree, and final classifier are fitted within that split's training rows. Only the selected columns are transferred to the test fold. Test rows are never filtered. This is repeated cross-validation with fold-specific preprocessing, not nested hyperparameter search.

Protocol A fits the cleaning steps once on the whole dataset and then cross-validates the final classifier on the cleaned data. This gives the cleaner access to future test labels. For instance-

filtering methods, it also removes future test cases and creates folds on a different sample. We report the resulting score changes descriptively. Protocol A folds are not paired with Protocol B folds, and their differences cannot isolate label leakage from evaluation-sample selection.

The comparison includes plain NB and C4.5, the original Algorithm 1–DT and Algorithm 2–NB methods, and the four versions of Algorithm 3. Two reference arms complete the attribution: Algorithm 1–NB changes only the final learner after filtering, and Algorithm 2–DT applies selection before another tree. N denotes filtering, A denotes attribute selection and weighting, and NA and AN denote the combined orders throughout the tables. Older CART experiments and later alternative filtering treatments are outside this paper’s comparison.

C. Measures and paired analysis

For each method and dataset, accuracy and macro-F1 are averaged over its 100 scored folds. Accuracy gives each fold equal weight. Macro-F1 first averages class F1 scores over classes present in that fold’s true labels or predictions, assigning zero when a class has no correct prediction. The reported summary then gives each of the ten datasets equal weight. It is not a pooled score over all benchmark instances.

We compare eight pairs using two-sided Wilcoxon signed-rank tests on the ten dataset-level accuracy averages: the two original hybrids against their respective baselines, each of the four combined methods against its baseline, and the two order comparisons. Exact permutation enumeration handles the small sample, including zero differences. Holm adjustment controls the family of eight comparisons at 0.05. This analysis was added during manuscript revision using the saved results; it is not a prospectively registered test family. Mean differences are reported in percentage points (pp). A nonsignificant test does not establish equivalence.

For each filtering step, we also record the fraction removed within each class, using that class’s training-row count as the denominator. This measure shows how deletion changes class representation without assuming that the discarded labels were incorrect.

V. RESULTS AND DISCUSSION

A. Combining the steps gives no clear advantage

Table II gives the benchmark averages, and Table III shows the individual datasets. Uncleaned C4.5 reaches 86.33% accuracy and 81.92% macro-F1. The two combined tree methods reach 81.21% and 80.89% accuracy, losses of 5.12 and 5.44 pp. Their macro-F1 scores also fall, to 75.76% and 73.93%. Adding attribute selection after filtering recovers only 0.18 pp relative to Algorithm 1’s 81.03% average accuracy.

For NB, attribute weighting alone gives the highest mean in this comparison, 77.35%, against 75.93% for plain NB. The combined methods reach 75.63% and 76.35%. Their changes relative to plain NB are -0.30 and $+0.42$ pp, and neither improves on Algorithm 2’s average. The reference arms help locate the loss: filtering followed by NB reaches 75.42%,

TABLE II
PROTOCOL B RESULTS (%), AVERAGED EQUALLY ACROSS TEN DATASETS

Method	Accuracy	Macro-F1
NB	75.93	69.80
C4.5	86.33	81.92
Algorithm 1	81.03	75.70
Algorithm 2	77.35	71.73
Instances first, NB	75.63	69.60
Instances first, DT	81.21	75.76
Attributes first, NB	76.35	69.91
Attributes first, DT	80.89	73.93

while selecting attributes before C4.5 leaves its mean almost unchanged at 86.19%.

Averages conceal substantial dataset variation. On Glass, instances-first NB falls from the plain NB baseline of 45.95% to 30.93%; instances-first C4.5 falls from 66.58% to 48.97%. On Contact lenses, the same NB combination improves from 73.00% to 83.00%, although that dataset has only 24 rows. These differences warrant inspection at the dataset level before describing a cleaner as helpful.

Table IV reports the paired accuracy comparisons. AN–DT is lower than the C4.5 baseline with raw $p = 0.014$, but its Holm-adjusted $p = 0.109$ exceeds the stated threshold. None of the eight comparisons is significant after adjustment. The observed mean losses remain relevant, but this benchmark does not establish a general disadvantage or advantage for every classification problem.

Changing from NA to AN raises mean NB accuracy by 0.71 pp and lowers mean tree accuracy by 0.32 pp. The corresponding raw p values are 0.275 and 0.922. We find no clear order advantage under these settings. This finding is limited to a single pass through the two steps, with fixed learner settings and the weights passed to the AN judge.

B. Cleaning before the split changes the score

Table V compares the two evaluation protocols. Algorithm 1 reaches 94.52% after full-data filtering, against 81.03% when the judge is refitted within each training fold. The apparent gain is 13.50 pp. Algorithm 2 changes much less, from 77.35% to 78.07%, or 0.72 pp. The combined tree methods also rise sharply under Protocol A, to 94.72% and 96.32%.

The procedures explain why these numbers answer different questions. Protocol B asks the final classifier to predict all held-out cases. Protocol A first uses the NB judge to choose which cases may reach the evaluation. The large increase is therefore compatible with removing cases the judge finds difficult, even when the downstream classifier does not improve on the full population. Attribute-only cleaning retains all rows, although its full-data selection still uses information from future test folds. Its smaller score change here should not be read as permission to select attributes before cross-validation.

C. Deletion changes class representation

The breast-cancer records show how uneven the removal can be. Across Protocol B folds, the plain NB judge removes an

TABLE III

PROTOCOL B ACCURACY BY DATASET (%). BASE IS THE UNCLEANNED CLASSIFIER; N IS INSTANCE FILTERING; A IS ATTRIBUTE SELECTION AND WEIGHTING; NA AND AN ARE THE TWO ALGORITHM 3 ORDERS. N-NB AND A-DT ARE REFERENCE ARMS.

Dataset	Final NB					Final C4.5				
	Base	N	A	NA	AN	Base	N	A	NA	AN
Breast cancer	72.12	72.68	72.43	71.28	72.53	74.84	71.77	73.91	71.77	72.08
Contact lenses	73.00	73.00	80.67	83.00	79.33	84.67	84.67	84.67	84.67	80.67
Diabetes	75.52	74.41	76.73	75.58	75.67	74.12	75.12	74.18	75.23	75.57
Glass	45.95	37.00	47.84	30.93	37.28	66.58	48.83	66.58	48.97	49.15
Segmentation	79.76	74.92	82.30	74.53	77.32	96.81	89.24	96.83	89.26	90.88
Iris	95.53	95.33	95.40	95.33	95.87	94.27	94.93	94.27	94.93	93.80
NSL-KDD	67.42	75.79	62.80	74.53	75.23	99.47	87.43	99.47	89.68	87.07
Soybean	90.09	89.00	89.40	85.67	86.66	92.71	88.71	92.26	87.89	90.98
Tic-tac-toe	69.83	71.89	70.68	70.03	68.07	84.74	74.43	84.74	74.55	73.17
Vote	90.06	90.13	95.24	95.45	95.49	95.08	95.13	95.03	95.13	95.54

TABLE IV

DATASET-LEVEL PAIRED ACCURACY TESTS. DIFFERENCES ARE FIRST METHOD MINUS SECOND; HOLM ADJUSTMENT COVERS ALL EIGHT COMPARISONS.

Comparison	Δ (pp)	p	Holm p
N-DT vs. DT	-5.30	0.055	0.383
A-NB vs. NB	+1.42	0.131	0.654
NA-DT vs. DT	-5.12	0.055	0.383
AN-DT vs. DT	-5.44	0.014	0.109
NA-NB vs. NB	-0.30	1.000	1.000
AN-NB vs. NB	+0.42	0.846	1.000
AN-DT vs. NA-DT	-0.32	0.922	1.000
AN-NB vs. NA-NB	+0.71	0.275	1.000

TABLE V

MEAN ACCURACY (%) UNDER THE TWO PROTOCOLS. BEFORE-CV FILTERING CHANGES THE EVALUATION SAMPLE; ITS SCORES ARE DIAGNOSTIC COMPARISONS.

Method	Fold-specific	Before CV	Difference
Algorithm 1	81.03	94.52	+13.50
Algorithm 2	77.35	78.07	+0.72
Alg. 1, NB final	75.42	91.93	+16.51
Alg. 2, DT final	86.19	86.52	+0.33
Instances first, NB	75.63	90.11	+14.48
Instances first, DT	81.21	94.72	+13.51
Attributes first, NB	76.35	91.08	+14.74
Attributes first, DT	80.89	96.32	+15.43

average of 50.63% of recurrence-event rows and 14.51% of no-recurrence-event rows. After attribute selection, the weighted judge removes 58.97% and 11.72%, respectively. These are mean class-specific removal rates, not percentages of the entire dataset.

The minority class loses a much larger share of its training examples in both paths. The records cannot establish whether those labels were wrong or whether NB failed on difficult but valid cases. They do show why a cleaner’s effect needs to be assessed through both classification scores and the training cases it discards.

VI. LIMITATIONS

The benchmark consists of the original study’s dataset family, with the file differences described above. It does not establish performance on modern datasets or on data with a known amount of label noise. In particular, the NSL-KDD C4.5 baseline is 99.47%, compared with 71.11% reported in 2014. That discrepancy remains unresolved. The current comparison uses shared data and splits, but should not be described as an exact replication of the earlier scores.

Rare classes limit the evaluation. Contact lenses has classes with four and five rows, and NSL-KDD contains classes with fewer than ten examples, including singleton classes. Some stratified folds therefore lack a class in training or testing. Repeating the splits does not supply new examples, and the fold-specific macro-F1 convention can average different class sets. These results cannot support reliable rare-class performance claims.

Although model fitting and supervised cleaning are confined to training rows in Protocol B, nominal category vocabularies come from the complete files. The estimate is conditional on those known vocabularies; it does not test deployment with unseen nominal values. No calibration or out-of-fold judging is used to validate the filter’s rejection decisions. The tests involve ten datasets and can miss small effects, especially after multiplicity adjustment. Finally, the Protocol A comparison cannot identify which evaluation procedure the 2014 study used.

VII. CONCLUSION

Algorithm 3 combines NB instance filtering and tree-based attribute weighting in either order. On ten datasets under fold-specific evaluation, both combined tree methods have lower mean accuracy and macro-F1 than uncleaned C4.5, while both combined NB methods trail attribute weighting alone. The paired tests provide no clear order advantage. The before-CV scores and class-specific removal rates support a practical recommendation: fit supervised cleaning within the training fold, score every held-out row, and report which classes lose training cases.

REFERENCES

- [1] D. M. Farid, L. Zhang, C. M. Rahman, M. A. Hossain, and R. Strachan, "Hybrid decision tree and naïve Bayes classifiers for multi-class classification tasks," *Expert Systems with Applications*, vol. 41, no. 4, pp. 1937–1946, 2014.
- [2] M. Hall, "A decision tree-based attribute weighting filter for naïve Bayes," *Knowledge-Based Systems*, vol. 20, no. 2, pp. 120–126, 2007.
- [3] C. E. Brodley and M. A. Friedl, "Identifying mislabeled training data," *Journal of Artificial Intelligence Research*, vol. 11, pp. 131–167, 1999.
- [4] G. C. Cawley and N. L. C. Talbot, "On over-fitting in model selection and subsequent selection bias in performance evaluation," *Journal of Machine Learning Research*, vol. 11, no. 70, pp. 2079–2107, 2010.
- [5] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *Journal of Machine Learning Research*, vol. 7, no. 1, pp. 1–30, 2006.
- [6] M. Tavallaei, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in *2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA)*. IEEE, 2009, pp. 1–6.
- [7] I. H. Witten, E. Frank, M. A. Hall, and C. J. Pal, *Data Mining: Practical Machine Learning Tools and Techniques*, 4th ed. Cambridge, MA: Morgan Kaufmann, 2016.